

TacZero: Training-Free Peg Insertion Using a General-Purpose Vision-Language Model with Tactile Feedback

Kazutoshi Tanaka¹

Abstract—Robots that autonomously determine their actions from language instructions and sensory observations could perform new contact-rich manipulation tasks without task-specific training or hand-designed rules. To perform these tasks, robots must infer how objects contact one another and move as a result, then select actions. For contact inference and action selection, prior approaches involve designing estimation models and tactile feedback control laws, or learning models for object-motion estimation, action-outcome prediction, and action selection from tactile data. Instead, we propose TacZero, which uses a pretrained general-purpose vision-language model (VLM) to interpret visual and tactile observations and select robot actions without additional tactile or manipulation training or task-specific rules for contact interpretation or action selection. TacZero provides the VLM with camera images, robot state, and three-axis tactile responses represented as numerical values overlaid on the images. From these observations and interaction history, the VLM generates commands specifying target end-effector positions and gripper opening or closing, which a low-level controller executes. In real-world cylindrical-peg insertion experiments, TacZero succeeded in 15 of 20 trials with numerical tactile input, compared with 10 of 20 without tactile input. This study provides a concrete starting point for further research on contact-rich manipulation using general-purpose VLMs and highlights challenges in pursuing this direction.

I. INTRODUCTION

Robots that autonomously determine their actions from language instructions and sensory observations could perform new contact-rich manipulation tasks [1], [2], such as peg insertion [3], without task-specific training or hand-designed rules. Contact-state estimation and reasoning about how contact constrains object motion can guide action selection in these tasks [4], [5]. Tactile sensing supports contact-state estimation by providing information about contact that is difficult to obtain from vision alone [6].

Prior work has developed model-based and learning-based approaches to contact-state estimation and action selection. Model-based approaches combine tactile measurements with geometric and contact constraints to estimate contact states [3] and determine robot motions through designed tactile feedback control laws [7]. Learning-based approaches train models to estimate object motion [3] or predict action outcomes from tactile data [8], or learn policies to select actions [9]. Other studies learn tactile representations that can be reused across perception tasks [10] or incorporate tactile

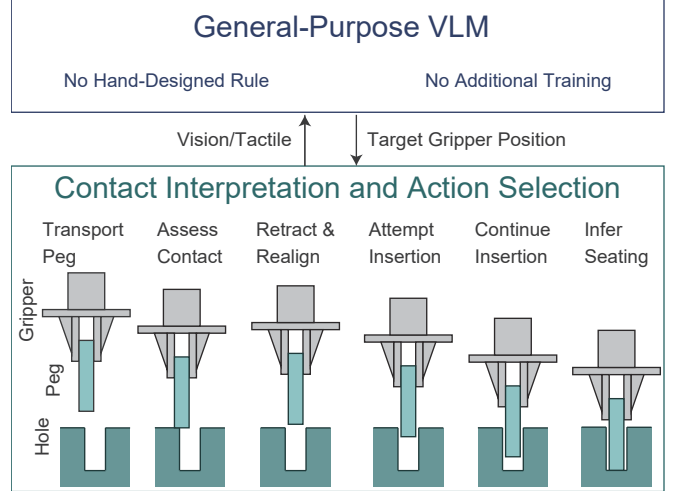


Fig. 1. TacZero uses a pretrained general-purpose VLM to interpret visual and tactile observations and select robot actions during peg insertion, without additional tactile or manipulation training or hand-designed task-specific rules for contact interpretation or action selection.

feedback into pretrained robot policies through fine-tuning on demonstrations from multiple manipulation tasks [11]. These approaches enable tactile-based manipulation through the design of models and control laws for contact-state estimation or action selection, or through learning from tactile data.

Vision-language models (VLMs) have enabled robotic manipulation without task-specific training [12], [13], [14]. These models use visual observations and language instructions to generate task plans [15], robot programs [16], and spatial targets or constraints for motion execution [13], [14]. These advances suggest that a pretrained general-purpose VLM could also interpret visual and tactile observations and select robot actions without additional tactile or manipulation training or hand-designed task-specific rules for contact interpretation or action selection.

To explore this possibility, we propose TacZero (Fig. 1), which uses a pretrained general-purpose VLM to interpret contact from visual and tactile observations and select robot actions, without additional tactile or manipulation training or hand-designed task-specific rules for contact interpretation or action selection. At each step, TacZero provides the VLM with the task instruction, visual and tactile observations, robot state, and the within-trial interaction history. The VLM outputs commands specifying target end-effector positions and gripper opening or closing, which a low-level controller executes. In real-world cylindrical-peg insertion experiments,

¹OMRON SINIC X Corporation, Bunkyo-ku, Tokyo 113-0033, Japan
kazutoshi.tanaka@sinicx.com

This work has been submitted to the IEEE for possible publication. Copyright may be transferred without notice, after which this version may no longer be accessible.

TacZero succeeded in 15 of 20 trials with numerical tactile input, compared with 10 of 20 without tactile input.

Our contributions are:

- We present a research direction in which a pretrained general-purpose VLM interprets visual and tactile observations and selects robot actions without additional tactile or manipulation training or hand-designed task-specific rules for contact interpretation or action selection. We provide a concrete starting point for future research and highlight challenges in pursuing this direction.
- We introduce TacZero, which provides a general-purpose VLM with a task instruction, visual and tactile observations, robot state, and interaction history, and executes the VLM’s output commands specifying target end-effector positions and gripper opening or closing.

II. RELATED WORK

A. Robotic Manipulation with Pretrained Language Models and Vision-Language Models

Researchers have proposed robotic manipulation methods in which pretrained language models (LMs) and VLMs generate policy code or action sequences before execution. Liang et al. proposed Code as Policies (CaP), which employs an LM to generate programs that process perception outputs, compute positions and trajectories, and invoke control application programming interfaces (APIs) [17]. Fang et al. proposed MOKA, in which a VLM selects grasping and interaction keypoints, pre-contact and post-contact waypoint regions, and motion attributes, which are then converted into robot motions using depth information and geometric processing [13]. Huang et al. proposed CoPa, in which a solver computes a target pose from VLM-generated spatial constraints between object parts, and the VLM also specifies subsequent numerical displacements and gripper opening [14]. Whereas these methods execute the generated programs or action sequences without calling the LM or VLM again for action selection during execution, TacZero calls the VLM after each action to determine the next action from fresh observations.

Other researchers have proposed methods that call LMs or VLMs again based on execution feedback to update plans or actions. Zhi et al. proposed COME-robot, which generates code that calls exploration, navigation, grasping, and placement functions and revises its plans using images and execution feedback [16]. Kwon et al. proposed a method that generates code to compute end-effector pose sequences and gripper commands from object geometry, then supplies observed object motion after trajectory execution to the LM for failure detection and replanning [18]. Wang et al. proposed Wonderful Team, in which VLM agents identify and refine target locations in images, specify action sequences, compute waypoints using generated functions for trajectory-based tasks, and support replanning based on execution results [15]. Nasiriany et al. proposed PIVOT, which presents candidate relative end-effector translations on the current image at

each action selection and iteratively updates the candidate distribution based on VLM selections [12]. Whereas these methods specify motions through operation-level functions, trajectory-generating code, image-space target coordinates, or motion-candidate selection, TacZero uses the VLM to specify the next end-effector target position numerically. Singh et al. proposed MALMM, which distributes planning, code generation, and coordination across multiple LM agents and replans using environmental observations obtained after intermediate execution steps [19]. In MALMM, the LM specifies numerical end-effector target positions within generated code; similarly, the VLM in TacZero determines numerical end-effector target positions.

None of the methods discussed in this subsection provide tactile observations to the LM or VLM. In contrast, TacZero provides a general-purpose VLM with tactile observations in addition to vision, robot state, and interaction history.

B. Tactile and Force Feedback for Robot Action Selection and Control

Researchers have proposed methods that integrate tactile feedback into robot action selection and refinement through additional training. Hao et al. proposed TLA, which uses sequential tactile images and learns corrective insertion actions from tactile-action-instruction data [20]. Zhang et al. proposed VTLa, a vision-language-action (VLA) model that uses supervised fine-tuning and preference learning to select corrective actions from visual and temporal tactile observations [21]. Bi et al. proposed VLA-Touch, which uses object-property descriptions from a pretrained tactile-language model for VLM-based planning and a trained tactile-conditioned controller to refine action sequences generated by a VLA. Although they also evaluated planning with tactile images supplied directly to a VLM, their main architecture assigns tactile-to-language interpretation and action refinement to dedicated models [22]. Van der Merwe et al. proposed TAP-VLA, which overlays tactile shear vectors onto camera images and fine-tunes a VLA on these images [23]. TacZero adopts a tactile overlay representation inspired by TAP-VLA. Unlike TAP-VLA, which fine-tunes a VLA on the overlaid images, TacZero supplies these images to a general-purpose VLM without additional training. Whereas these approaches use models trained for tactile-based action selection or refinement, TacZero uses a single general-purpose VLM to interpret visual and tactile observations and select robot actions without additional tactile or manipulation training.

Other researchers have proposed methods that combine a VLM with force-based control and search procedures without additional training. Yajima et al. proposed a method that employs a VLM to identify the mating hole and determine the relative orientation, and executes insertion using predefined force-based control and search procedures [24]. Whereas the VLM in their method identifies the mating hole and determines the relative orientation, leaving insertion motions to predefined control and search procedures, the VLM in

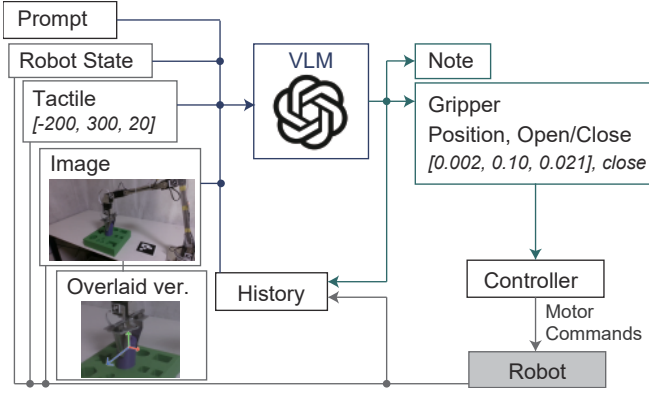


Fig. 2. Overview of TacZero. Given the task instruction, visual and tactile observations, robot state, and interaction history, the VLM outputs a target end-effector position, a gripper opening or closing command, and a note containing a short observation and reason. The controller executes the command, and new observations are fed back to the VLM.

TacZero determines the next end-effector target position from observations at each step.

III. TACZERO

A. Task Setting

We consider robotic peg insertion specified by a natural-language instruction. The instruction describes the peg and the desired insertion outcome without specifying target coordinates or a sequence of actions. The robot must insert the peg into the corresponding hole and maintain its grasp until insertion is complete. We do not perform additional tactile or manipulation training or update the VLM’s parameters.

The robot has access to camera images, robot state, and tactile responses along the X, Y, and Z axes of the sensor coordinate frame, measured by a sensor mounted on the inner surface of one gripper finger. The VLM is not provided with explicit dimensions of the peg or the hole. Given the instruction and the interaction history available up to the current step, the VLM specifies the next target position of the end effector and a gripper opening or closing command. The end-effector orientation remains fixed throughout VLM-controlled execution.

A trial comprises a sequence of observation and action steps. It ends when the VLM calls `report_completion`, the experimenter stops execution, or a predefined execution limit or safety stop is triggered. An insertion is successful when the peg’s lower end is near the bottom of the specified hole. The experimenter judges success through visual inspection, independently of the VLM’s completion report. If the experimenter observes that the peg has slipped out of the grasp or judges that slippage within the grasp prevents further insertion to the required depth, they stop the trial and record it as a failure. No interaction history or human-assigned outcome labels from previous trials are provided to the VLM.

B. Overview

TacZero uses a pretrained general-purpose VLM to interpret visual and tactile observations and select robot actions,

without additional tactile or manipulation training or hand-designed task-specific rules for contact interpretation or action selection. Fig. 2 illustrates the overall framework. At each step, the VLM receives the task instruction, the current camera image, tactile responses, robot state, and the interaction history from the same trial. Tactile responses are presented as numerical values or vectors overlaid on the camera image. The VLM outputs the next end-effector target position and gripper opening or closing command, together with a note containing a short observation and reason. A low-level controller executes the command, and the resulting observations are supplied to the VLM at the next step. The interaction history includes observations, commands, notes, and execution results from the same trial, enabling the VLM to relate previous actions to subsequent changes in visual and tactile observations.

C. VLM Inputs and Action Selection

At each step, the VLM receives the task instruction, camera image, tactile responses, and robot state. The image provides information about the spatial relationships between objects and the end effector, while tactile responses provide complementary information about contact that may be difficult to discern from images alone. Robot state includes the end-effector position and orientation, joint and gripper encoder counts, and motor currents. We also provide camera calibration parameters and coordinate transformations to relate image observations to the robot reference frame. We instruct the VLM to specify target end-effector positions as absolute XYZ coordinates in this frame, expressed in meters, and provide the workspace bounds and execution limits. Observations also include acquisition timestamps.

We instruct the VLM to include a short observation and reason in the note accompanying the next end-effector target position and gripper opening or closing command. TacZero retains the command and its accompanying note in the within-trial interaction history together with subsequent observations and execution results, and provides this history to the VLM at the next step. This allows the VLM to relate previous actions and their stated reasons to subsequent changes in visual and tactile observations when selecting the next action. This interleaving of verbal reasoning, actions, and observations is similar to that used in ReAct [25].

D. Implementation Details

Fig. 3 shows the experimental apparatus. We use a six-degree-of-freedom arm with the same joint-axis configuration, motors, and link lengths as the ALOHA follower arm [26]. The arm is equipped with a two-finger gripper with flat inner contact surfaces, one of which serves as the mounting surface for a uSkin uSPa44 tactile sensor (XELA Robotics). The sensor consists of 16 taxels arranged in a 4×4 array, each measuring responses along the X, Y, and Z axes of the sensor coordinate frame. An overhead RGB-D camera (RealSense D435, Intel) observes the workspace. RGB images are supplied at a resolution of 1920×1080 pixels.

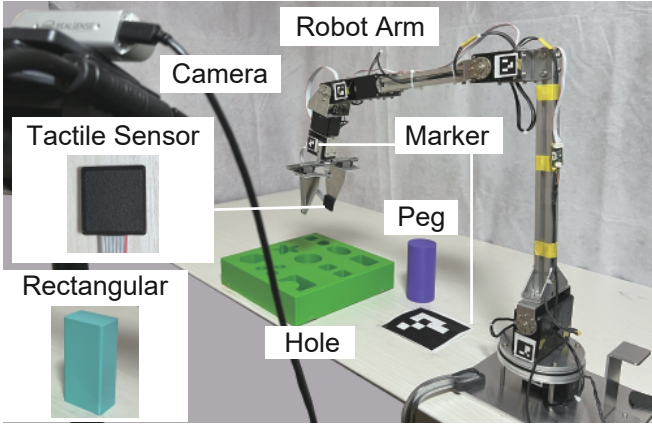


Fig. 3. Peg-insertion setup with a robot arm, an overhead camera, and a tactile sensor mounted on one gripper finger. Insets show the tactile sensor and rectangular-prism peg.

A stationary ArUco marker [27] defines the reference coordinate frame. We estimate the pose of a hand-mounted ArUco marker from the camera image and transform it into the tool center point (TCP) pose in the reference frame using calibrated coordinate transformations. We use the TCP as the reference point for specifying target end-effector positions.

Before motor initialization and grasp preparation in each trial, we compute a baseline for each taxel as the component-wise median of 30 unloaded tactile frames. We keep this baseline fixed throughout the trial. Let $\mathbf{s}_{j,t} \in \mathbb{R}^3$ denote the XYZ response of taxel j at time t , and let $\mathbf{b}_j \in \mathbb{R}^3$ denote its baseline. We compute the aggregate tactile response $\mathbf{f}_t^S \in \mathbb{R}^3$ in the sensor coordinate frame S as

$$\mathbf{f}_t^S = \sum_{j=1}^{16} (\mathbf{s}_{j,t} - \mathbf{b}_j), \quad (1)$$

where the sum includes all 16 taxels. We express these responses in sensor counts rather than calibrated force units. For each image, we select the latest tactile sample received at or before the image capture time.

We present the aggregate tactile response to the VLM in two forms: numerical values or vectors overlaid on the RGB image. In the numerical representation, we provide the three components of $\mathbf{f}_t^S$ alongside the unmodified RGB image. In the overlay representation, we render the signed responses along the sensor X, Y, and Z axes as red, green, and blue arrows, respectively. Let E , 0 , and C denote the end-effector, stationary reference, and camera frames, respectively, and let R_{AB} denote the rotation matrix that maps vector components from frame B to frame A . For each sensor axis $i \in \{X, Y, Z\}$, let $f_{i,t}^S$ denote the corresponding component of $\mathbf{f}_t^S$. We transform each axis component into the camera frame as

$$\mathbf{v}_{i,t}^C = R_{C0} R_{0E,t} R_{ES} (f_{i,t}^S \mathbf{e}_i), \quad (2)$$

where $\mathbf{e}_i$ is the unit vector along axis i in the sensor frame. We determine R_{ES} from the sensor mounting, obtain $R_{0E,t}$ from the measured end-effector orientation, and obtain R_{C0}

from the pose of the stationary reference marker in the camera frame.

We then project these camera-frame vectors onto the RGB image, using the projected TCP position as the origin of each arrow. Let $\mathbf{p}_{\text{TCP},t}^C$ denote the measured TCP position in the camera frame, and let π denote the camera projection function incorporating intrinsic parameters and distortion coefficients. We use a local linear approximation of the camera projection at the TCP position to map each three-dimensional tactile-response vector to a two-dimensional image arrow. We compute the two-dimensional displacement from the origin to the endpoint of each arrow, before applying length limits, as

$$\Delta \mathbf{u}_{i,t} = \alpha J_\pi (\mathbf{p}_{\text{TCP},t}^C) \mathbf{v}_{i,t}^C, \quad (3)$$

where $\Delta \mathbf{u}_{i,t}$ is expressed in pixels and J_π is the Jacobian of π evaluated at the TCP position, computed using central differences. We use a fixed display scale of $\alpha = 1.875 \times 10^{-5}$ m/count to convert tactile responses into lengths for visualization. We limit arrow lengths to 240 pixels and clip them at the image boundary. We inform the VLM whether any arrows have been clipped. The arrow origin is a display anchor, not an estimated contact location. Projection onto the image plane and arrow clipping can reduce the directional and magnitude information conveyed by the overlay, whereas the numerical representation preserves all three aggregate components without projection or clipping.

We use GPT-6 Astra (gpt-6-astra) as the primary VLM, accessed through the API with the reasoning effort set to medium. At each step, Astra receives the observations and interaction history and returns a structured tool call. The tool `move_to(x, y, z, gripper, note)` accepts an absolute TCP target position in the reference frame, a gripper opening or closing command, and a note containing a short observation and reason. The VLM can specify each target coordinate as a numerical value in meters or as null. A null coordinate leaves the corresponding target coordinate unchanged. Before the first command, we initialize the target position from the measured TCP position. When all coordinates are null and a gripper command is specified, only the gripper is actuated. When a command specifies both translation and gripper opening or closing, the controller executes the translation first, followed by the gripper operation. Calling `report_completion(note)` ends the trial and records the VLM's final note, which may report completion or state that the task remains incomplete.

During each movement, the controller repeatedly uses the marker-based TCP pose to update the position and orientation errors relative to the fixed target. It computes translational and angular velocity commands from these errors and converts them into joint velocity commands through inverse differential kinematics. The translational velocity command is proportional to the position error and capped at 20 mm/s, decreasing as the TCP approaches the target. The target end-effector orientation is fixed so that the end effector points downward relative to the reference frame. Each movement runs for the initial TCP-to-target distance divided by 20

mm/s, plus 3 seconds, with timing starting when the first velocity-command transmission completes. Reaching this time limit ends the movement regardless of whether the target has been attained. The controller returns execution results to the VLM, including execution duration, whether the time limit was reached, whether the target was attained, and final position and orientation errors when available. The VLM receives these results together with fresh observations for selecting the next action. We limit execution under VLM control to 100 action commands, 600 s of accumulated actuator-invocation time, or 1800 s of wall time, whichever is reached first.

IV. EXPERIMENTS

A. Setup

We evaluate TacZero through experiments addressing four questions: (1) Can a pretrained general-purpose VLM infer contact conditions from visual and tactile observations and select actions to perform peg insertion without additional tactile or manipulation training or hand-designed task-specific rules? (2) Does tactile input improve insertion performance over a no-tactile baseline? (3) How does insertion performance differ between numerical and overlay representations of the tactile responses? (4) Can TacZero also perform insertion with another VLM and a different peg shape?

To facilitate reproduction of the setup, we use cylindrical and rectangular-prism pegs with their corresponding hole fixtures from the Functional Manipulation Benchmark (FMB) [28]. We select the Large size and Short length for both peg shapes. The hole fixture remains in the same location throughout each comparison, with a non-slip mat underneath to prevent movement. For cylindrical-peg insertion, the instruction is: “Fully insert the purple cylinder into the central hole of the green fixture until it reaches the bottom. Maintain your grasp until insertion is complete.” For rectangular-prism peg insertion, the instruction is: “Fully insert the light blue rectangular prism into its corresponding hole until it reaches the bottom. Maintain your grasp until insertion is complete.”

Using GPT-6 Astra (`gpt-6-astra`), we compare three input conditions: a no-tactile baseline, which receives camera images and robot state without tactile input; TacZero-Numbers, which additionally receives aggregate three-axis tactile responses as numerical text; and TacZero-Overlay, which receives these responses as arrows overlaid on the camera images. We conduct 20 independent cylindrical-peg insertion trials per condition. Both tactile conditions use the same baseline-corrected three-axis responses, presented as numerical text or image overlays. We additionally evaluate TacZero-Numbers in 10 independent rectangular-prism peg insertion trials using Astra and in 10 independent cylindrical-peg insertion trials using Claude Fable 5.1 (`claude-fable-5-1`) at high reasoning effort. All Astra trials use medium reasoning effort. Across these experiments, we use the same robot-state inputs, translation controller, execution limits, and history-inclusion scheme, which retains all previous observation images from the same trial. The task

TABLE I
ACTION COUNTS, EXECUTION TIME, TOKEN USAGE, AND ESTIMATED API COST.

	None	Num.	Overlay	Box	Fable
Trials	20	20	20	10	10
Actions	19.9	24.3	23.2	23.0	8.9
<i>Mean time per trial</i>					
Motion (s)	94	110	105	99	52
API (s)	177	239	224	223	277
Time (s)	271	349	329	322	330
Tokens (M)	0.952	1.817	1.731	1.544	0.419
<i>Estimated cost (USD)</i>					
Cost (\$)	12.02	22.84	21.41	19.43	4.91
Total (\$)	240.49	456.83	428.13	194.35	49.05

None, Num., and Overlay denote the no-tactile baseline, TacZero-Numbers, and TacZero-Overlay, respectively, for cylindrical-peg insertion with Astra. Box and Fable denote rectangular-prism insertion with Astra and cylindrical-peg insertion with Fable, respectively, both using TacZero-Numbers.

instruction is the same for all trials with the same peg shape. The base prompt includes the instruction: “Use exactly one tool each turn and include a short observation and reason in note.” In the tactile conditions, we append “considering tactile input when available” to this instruction.

Before each trial, the experimenter returns the arm to its initial pose and manually places the peg in approximately the same starting region. To focus on TacZero’s behavior during peg insertion, the robot follows a fixed sequence to grasp and lift the peg before VLM control begins. After the experimenter confirms the grasp, the VLM starts controlling insertion.

To examine whether a general-purpose VLM can infer contact conditions and select actions for peg insertion, we analyze the observations and reasons recorded in its notes alongside executed actions and visual and tactile observations across trials. To assess whether tactile input improves insertion performance and how its representation affects the results, we compare human-judged success rates and failure categories across the three input conditions. The experimenter assigns each failed trial to one of three mutually exclusive categories: no entry, jamming after entry, or grasp slippage. Here, grasp slippage refers to displacement of the peg relative to the gripper fingers caused by contact forces from the hole fixture. Finally, we report action counts, execution time, API waiting time, token usage, and estimated API cost.

B. Results

Table I summarizes action counts, execution time, API waiting time, token usage, and estimated API cost. All values except the trial counts and total costs are means per trial. Time is the sum of action execution and API waiting, excluding preparation, image uploads, and other processing overhead. Token usage includes input and output tokens, and costs are estimated from published rates, excluding calls with unreported usage. API waiting time exceeded action execution time in all conditions. For cylindrical-peg insertion with Astra under the TacZero-Numbers condition, the combined

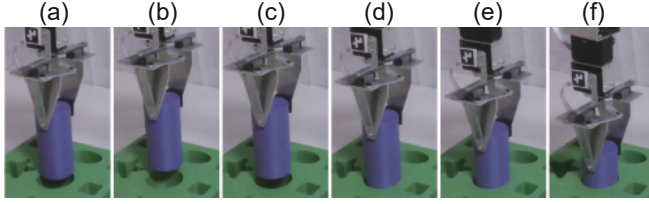


Fig. 4. Successful TacZero-Numbers trial 3: (a) entrance contact, (b) retraction, (c) realignment, (d) entry, (e) insertion, and (f) completion report.

TABLE II
INSERTION OUTCOMES BY CONDITION.

	None	Num.	Overlay	Box	Fable
Trials	20	20	20	10	10
Successes	10	15	11	5	2
Success (%)	50	75	55	50	20
<i>Failures (trials)</i>					
No entry	2	2	3	2	7
Jamming	6	3	6	3	0
Slip	2	0	0	0	1

None, Num., and Overlay denote the no-tactile baseline, TacZero-Numbers, and TacZero-Overlay, respectively, for cylindrical-peg insertion with Astra. Box and Fable denote rectangular-prism insertion with Astra and cylindrical-peg insertion with Fable, respectively, both using TacZero-Numbers.

action-execution and API-waiting time averaged 349 s per trial, of which 239 s was spent waiting for the API. The entire insertion phase, including image processing, uploads, and other overhead, averaged 446 s. Fable used fewer actions and had a lower estimated API cost. The estimated API cost across all 80 trials was USD 1,369.85.

Fig. 4 shows a successful cylindrical-peg insertion trial with TacZero-Numbers. At the beginning of the trial, the VLM described the peg as held upright beside the fixture and selected a higher target end-effector position, citing the need for clearance before lateral motion. Although the hole’s numerical coordinates were not supplied, the VLM then selected a target end-effector position above the estimated hole center, citing the camera images and calibration information and noting that the lift had been visually confirmed. After approaching the hole, the VLM specified a higher target end-effector position. In its note, it described stalled downward motion and an increased tactile response in the vertical direction of the robot reference frame and gave entrance contact rather than bottom contact as the reason for retracting (a). It then specified a target end-effector position offset in XY above the opening. Its note stated that the images and TCP measurements indicated clearance and that the tactile response had subsided (b). Panel (c) shows a later entry attempt. After the peg entered the hole, the VLM specified lower target end-effector positions, citing visible insertion progress and the absence of an increase in the vertical tactile response in its notes (d, e). It then specified lower target positions in smaller increments and reported completion. In its final note, it described little visible change in insertion depth and an increased vertical tactile response, identifying bottom contact as the reason for reporting completion (f).

Table II presents the success rates. In cylindrical-peg

insertion with Astra, TacZero-Numbers succeeded in 15 of 20 trials (75%), compared with 10 of 20 (50%) for the no-tactile baseline and 11 of 20 (55%) with TacZero-Overlay. TacZero-Numbers yielded the highest observed success rate, whereas the success rate with TacZero-Overlay was close to that of the no-tactile baseline. These results suggest that tactile input can improve insertion performance, but its benefit may depend on how it is represented to the VLM.

TacZero-Numbers also performed successful insertions with another peg shape and another VLM. With Astra, it succeeded in 5 of 10 rectangular-prism peg insertion trials (50%). With Fable, it succeeded in 2 of 10 cylindrical-peg insertion trials (20%). The rectangular-prism failures comprised two trials without hole entry and three with jamming after entry. The Fable failures comprised seven trials without hole entry and one with grasp slippage.

In the cylindrical-peg comparison, failures to enter the hole occurred in 2 trials with the no-tactile baseline, 2 with TacZero-Numbers, and 3 with TacZero-Overlay. Failures due to jamming after entry occurred in 6, 3, and 6 trials, respectively. Two further failures with the no-tactile baseline were attributed to grasp slippage. Thus, TacZero-Numbers had fewer jamming failures, while failures to enter the hole were similar across conditions. In its notes, the VLM frequently referred to vertical tactile responses, whereas references to horizontal responses were limited even in trials judged by the experimenter to have failed due to jamming during insertion.

In both the TacZero-Numbers and TacZero-Overlay conditions, some failed trials ended before the peg entered the hole, with the VLM explicitly stating in its final note that the task was incomplete. In other failed trials in both conditions, the VLM stated in its final note that the peg had reached the bottom and reported completion, although the experimenter judged that the peg had jammed before reaching the required insertion depth.

V. DISCUSSION

In our experiments, TacZero-Numbers yielded a higher peg-insertion success rate than the no-tactile baseline. The general-purpose VLM retracted and reinserted the peg, citing tactile responses in its notes. These results show that TacZero can perform peg insertion by using a general-purpose VLM to interpret visual and tactile observations and select robot actions, without additional tactile or manipulation training or hand-designed rules for contact interpretation or action selection. TacZero provides a concrete baseline for developing and comparing future methods. The following limitations highlight challenges for further research on this approach.

One limitation concerns how we presented observations to the VLM. We used a single camera viewpoint, leaving some aspects of peg–hole alignment and contact visually ambiguous. In its notes, the VLM frequently referred to vertical tactile responses but infrequently referred to horizontal responses. This tendency may depend in part on the fixed end-effector orientation and the textual descriptions of coordinate-frame relationships provided in our setup. We compared numerical and overlay representations only

within this setup, so the advantage of TacZero-Numbers over TacZero-Overlay may not extend to other viewpoints or coordinate descriptions. One direction for future work is therefore to jointly design camera viewpoints, coordinate descriptions, and tactile representations to help general-purpose VLMs better interpret visual and tactile observations and select robot actions.

A similar limitation concerns how we specified the goal and action constraints. We provided the goal in text without an image of the completed task, so the VLM could not compare its current view with a visual reference of completion. In some failed trials, the VLM reported completion even though the experimenter judged that the peg had not entered the hole or had jammed before reaching the required insertion depth. We left the decision to continue or terminate an attempt to the VLM, which sometimes abandoned attempts while further corrections remained possible. We also required the VLM to maintain its grasp, preventing regrasping after slippage. Another direction for future work is to design human-VLM interaction, including how goals and instructions are conveyed, so that the VLM's actions better reflect human intent.

Another limitation is the latency of VLM responses. In our implementation, each movement toward a target end-effector position at a fixed orientation was followed by an observation and a wait for the VLM's next command. As a result, increases in tactile responses or slippage of the peg relative to the gripper fingers during motion or while waiting for a response could not trigger immediate stopping or motion adjustments. Beyond movements toward individual target positions at a fixed end-effector orientation, future work should support a broader range of motion trajectories and changes in end-effector orientation. A further direction is to design a hierarchical system that asynchronously combines VLM-based contact interpretation and action selection with rapid monitoring and stopping and tactile feedback control.

VI. CONCLUSION

We proposed TacZero, which uses a pretrained general-purpose VLM to interpret visual and tactile observations and select robot actions without additional tactile or manipulation training or task-specific rules for contact interpretation or action selection. In real-world cylindrical-peg insertion experiments, TacZero succeeded in 15 of 20 trials with numerical tactile input, compared with 10 of 20 without tactile input. This study provides a concrete starting point for further research on this approach and highlights challenges in pursuing this direction.

ACKNOWLEDGMENTS

The authors used OpenAI Codex with GPT-5.6 Sol and GPT-6 Astra to assist in drafting text throughout the manuscript and in developing experimental software that incorporated pre-existing code.

REFERENCES

- [1] M. Suomalainen, Y. Karayiannidis, and V. Kyrki, "A survey of robot manipulation in contact," *Robotics and Autonomous Systems*, vol. 156, p. 104224, 2022.
- [2] B. Tang, M. A. Lin, I. A. Akinola, A. Handa, G. S. Sukhatme, F. Ramos, D. Fox, and Y. S. Narang, "IndustReal: Transferring contact-rich assembly tasks from simulation to reality," in *Robotics: Science and Systems (RSS)*, 2023.
- [3] S. Kim and A. Rodriguez, "Active extrinsic contact sensing: Application to general peg-in-hole insertion," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2022, pp. 10 241–10 247.
- [4] S. Kim, D. K. Jha, D. Romeres, and A. Rodriguez, "Simultaneous tactile estimation and control of extrinsic contact," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2023, pp. 12 563–12 569.
- [5] X. Cheng, E. Huang, Y. Hou, and M. T. Mason, "Contact mode guided motion planning for quasidynamic dexterous manipulation in 3D," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2022, pp. 2730–2736.
- [6] Q. Li, O. Kroemer, Z. Su, F. F. Veiga, M. Kaboli, and H. Ritter, "A review of tactile information: Perception and action through touch," *IEEE Transactions on Robotics (T-RO)*, vol. 36, no. 6, pp. 1619–1634, 2020.
- [7] Q. Li, C. Schürmann, R. Haschke, and H. Ritter, "A control framework for tactile servoing," in *Robotics: Science and Systems (RSS)*, 2013.
- [8] S. Tian, F. Ebert, D. Jayaraman, M. Mudigonda, C. Finn, R. Calandra, and S. Levine, "Manipulation by feel: Touch-based control with deep predictive models," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2019, pp. 818–824.
- [9] S. Dong, D. K. Jha, D. Romeres, S. Kim, D. N. Nikovski, and A. Rodriguez, "Tactile-RL for insertion: Generalization to objects of unknown geometry," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2021, pp. 6437–6443.
- [10] C. Higuera, A. Sharma, C. K. Bodduluri, T. Fan, P. Lancaster, M. Kalakrishnan, M. Kaess, B. Boots, M. Lambeta, T. Wu, and M. Mukadam, "Sparsh: Self-supervised touch representations for vision-based tactile sensing," in *Conference on Robot Learning (CoRL)*, ser. Proceedings of Machine Learning Research, vol. 270. PMLR, 2025, pp. 885–915.
- [11] J. Jones, O. Mees, C. Sferrazza, K. Stachowicz, P. Abbeel, and S. Levine, "Beyond sight: Finetuning generalist robot policies with heterogeneous sensors via language grounding," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2025, pp. 5961–5968.
- [12] S. Nasiriany, F. Xia, W. Yu, T. Xiao, J. Liang, I. Dasgupta, A. Xie, D. Driess, A. Wahid, Z. Xu, Q. Vuong, T. Zhang, T.-W. E. Lee, K.-H. Lee, P. Xu, S. Kirmani, Y. Zhu, A. Zeng, K. Hausman, N. Heess, C. Finn, S. Levine, and B. Ichter, "PIVOT: Iterative visual prompting elicits actionable knowledge for VLMs," in *International Conference on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 235. PMLR, 2024, pp. 37 321–37 341.
- [13] K. Fang, F. Liu, P. Abbeel, and S. Levine, "MOKA: Open-world robotic manipulation through mark-based visual prompting," in *Robotics: Science and Systems (RSS)*, 2024.
- [14] H. Huang, F. Lin, Y. Hu, S. Wang, and Y. Gao, "CoPa: General robotic manipulation through spatial constraints of parts with foundation models," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2024, pp. 9488–9495.
- [15] Z. Wang, R. Shen, and B. Stadie, "Wonderful Team: Zero-shot physical task planning with visual LLMs," *Transactions on Machine Learning Research (TMLR)*, 2025.
- [16] P. Zhi, Z. Zhang, Y. Zhao, M. Han, Z. Zhang, Z. Li, Z. Jiao, B. Jia, and S. Huang, "Closed-loop open-vocabulary mobile manipulation with GPT-4V," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2025, pp. 4761–4767.
- [17] J. Liang, W. Huang, F. Xia, P. Xu, K. Hausman, B. Ichter, P. Florence, and A. Zeng, "Code as Policies: Language model programs for embodied control," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2023, pp. 9493–9500.
- [18] T. Kwon, N. D. Palo, and E. Johns, "Language models as zero-shot trajectory generators," *IEEE Robotics and Automation Letters (RA-L)*, vol. 9, no. 7, pp. 6728–6735, 2024.

- [19] H. Singh, R. J. Das, M. Han, P. Nakov, and I. Laptev, "MALMM: Multi-agent large language models for zero-shot robotic manipulation," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2025, pp. 20 386–20 393.
- [20] P. Hao, C. Zhang, D. Li, X. Cao, X. Hao, S. Cui, and S. Wang, "TLA: Tactile-language-action model for contact-rich manipulation," *Robot Learning (Robot Learn.)*, vol. 3, no. 1, 2026, Art. no. 0001.
- [21] C. Zhang, P. Hao, X. Cao, X. Hao, S. Cui, and S. Wang, "VTLA: Vision-tactile-language-action model with preference learning for insertion manipulation," *Biomimetic Intelligence and Robotics (BIROB)*, 2026, Art. no. 100333.
- [22] J. Bi, K. Y. Ma, C. Hao, M. Z. Shou, and H. Soh, "VLA-Touch: Enhancing vision-language-action model with dual-level tactile feedback," *IEEE Robotics and Automation Letters (RA-L)*, vol. 11, no. 7, pp. 8487–8494, 2026.
- [23] M. Van der Merwe, M. L. Shehab, J. Lee, Y. Wi, Y. Dai, D. Berenson, and N. Fazeli, "TAP-VLA: Tactile annotation prompting for vision language action models," arXiv:2606.29089, 2026.
- [24] M. Yajima, K. Ota, A. Kanezaki, and R. Kawakami, "Zero-shot peg insertion: Identifying mating holes and estimating SE(2) poses with vision-language models," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2025, pp. 14 907–14 914.
- [25] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations (ICLR)*, 2023.
- [26] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, "Learning fine-grained bimanual manipulation with low-cost hardware," in *Robotics: Science and Systems (RSS)*, 2023.
- [27] S. Garrido-Jurado, R. Muñoz-Salinas, F. J. Madrid-Cuevas, and M. J. Marín-Jiménez, "Automatic generation and detection of highly reliable fiducial markers under occlusion," *Pattern Recognition (PR)*, vol. 47, no. 6, pp. 2280–2292, 2014.
- [28] J. Luo, C. Xu, F. Liu, L. Tan, Z. Lin, J. Wu, P. Abbeel, and S. Levine, "FMB: A functional manipulation benchmark for generalizable robotic learning," *The International Journal of Robotics Research (IJRR)*, vol. 44, no. 4, pp. 592–606, 2025.